

期末复习 PPT 完整模拟卷 02（题目与完整解析）

卷面说明

考试时间：150 分钟

总分：100 分

题型：计算、构造、翻译与简答

说明：本卷题面给出了全部文法、控制流、数据流和目标机条件，不需要查阅其他文件。

A 卷题目

一、词法分析与 LL(1) 分析（20 分）

（一）词法分析（10 分）

给定正则表达式：

$(a|b)^*a$

1. 使用 Thompson 构造法构造等价 NFA，写出起始状态、接受状态和全部转移。 2. 使用子集构造法构造 DFA，写出每个 DFA 状态对应的 NFA 状态集合及转移表。 3. 最小化 DFA，写出最终状态集合和转移表。

（二）LL(1) 分析（10 分）

给定文法，p、r、t、u 均为终结符：

$A \rightarrow B C$
 $B \rightarrow B p \mid r$
 $C \rightarrow t \mid u$

1. 消除直接左递归。 2. 计算所有非终结符的 FIRST 和 FOLLOW。 3. 构造预测分析表。 4. 判断改写后的文法是否为 LL(1)，并写出依据。

二、LR 分析与语义分析（20 分）

（一）SLR 分析（12 分）

给定增广文法：

(0) $S' \rightarrow S$
(1) $S \rightarrow C C$
(2) $C \rightarrow c C$
(3) $C \rightarrow d$

1. 构造完整规范 LR(0) 项集族，并写出全部 GOTO 转移。 2. 计算 FOLLOW(S) 和 FOLLOW(C)。 3. 构造完整 SLR ACTION/GOTO 表。 4. 判断是否存在移进-规约或规约-规约冲突。

（二）L 属性 SDD（8 分）

给定声明文法：

$D \rightarrow T L$
 $T \rightarrow \text{int} \mid \text{float}$
 $L \rightarrow \text{id } L'$
 $L' \rightarrow , \text{id } L' \mid \text{epsilon}$

设计一个 L 属性 SDD，将每个 id 及其类型加入符号表；再写出分析输入 float x, y 时的属性传递顺序和最终符号表。

三、中间代码、控制流与 SSA（20 分）

给定程序片段，假定数组元素宽度为 4：

```
while (i < n) {
    if (a[i] > 0)
        sum = sum + a[i];
    i = i + 1;
}
```

1. 生成完整三地址码，要求显式写出标签和跳转目标。 2. 划分基本块并写出 CFG 边。 3. 把程序改写成 SSA，要求正确放置 i 和 sum 的 phi 函数。 4. 说明两个 phi 函数在退出 SSA 时应在哪些控制流边上插入复制。

四、运行时环境与目标代码生成（20 分）

（一）运行时环境（10 分）

函数调用如下：

```
int add(int a, int b) {
    int t = a + b;
    return t;
}

int main() {
    int x = 3;
    int y = 4;
    int z = add(x, y);
    return z;
}
```

1. 写出 add 的活动记录中至少应包含的内容。 2. 按“参数从右到左压栈”的约定，写出调用 add(x, y) 时的压栈顺序。 3. 说明 BP、SP、返回地址和返回值在调用/返回过程中的作用。 4. 比较引用计数与标记-清扫回收，指出引用计数无法处理的结构。

（二）目标代码生成（10 分）

三地址码如下，目标机只有寄存器 R0、R1：

```
t1 = a - b
t2 = t1 + c
x = t2
```

目标机指令为：

```
LD R, x
ST x, R
ADD Rd, Rs
SUB Rd, Rs
```

其中 ADD Rd, Rs 表示 $Rd = Rd + Rs$ ，SUB Rd, Rs 表示 $Rd = Rd - Rs$ 。

1. 生成一组完整目标代码。 2. 逐条写出寄存器描述符变化。 3. 写出最终地址描述符中 x 的位置。

五、数据流分析、支配与自然循环（20 分）

CFG 和基本块定值如下：

```
Entry -> B1
B1 -> B2
B2 -> B3
B2 -> B4
B3 -> B2
B4 -> Exit
```

B1:

```
d1: x = 1
d2: y = 2
```

B2:

```
d3: z = x + y
```

B3:

```
d4: x = z
```

B4:

```
use(z)
```

1. 写出到达定值分析的方向、交汇运算和传递方程。
2. 写出每个基本块的 gen/kill。
3. 从空集初始化，计算各基本块最终 IN/OUT。
4. 计算各基本块的支配集合和直接支配者。
5. 找出回边，并写出对应自然循环。

B 按考试标准重写答案

一、词法分析与 LL(1) 分析答案

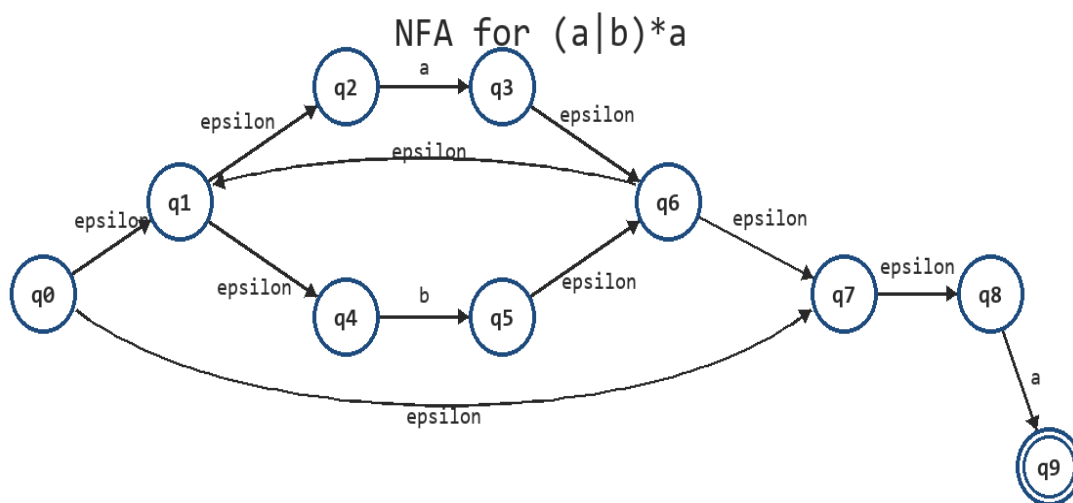
(一) 词法分析

正则表达式是：

$(a|b)^*a$

它表示任意个 a 或 b 后面再接一个 a，也就是所有以 a 结尾的 {a, b} 串。

Thompson NFA 如图。起始状态为 q0，接受状态为 q9。



Thompson NFA for $(a|b)^*a$

NFA 转移表：

起点	边标号	终点	含义	
q0	epsilon	q1	进入 $(a b)^*$	
q0	epsilon	q7	* 重复 0 次	
q1	epsilon	q2	选择 a 分支	
q1	epsilon	q4	选择 b 分支	
q2	a	q3	匹配 a	
q4	b	q5	匹配 b	
q3	epsilon	q6	分支结束	
q5	epsilon	q6	分支结束	
q6	epsilon	q1	* 回到循环	
q6	epsilon	q7	* 退出循环	
q7	epsilon	q8	连接最后一个 a	
q8	a	q9	匹配末尾 a	

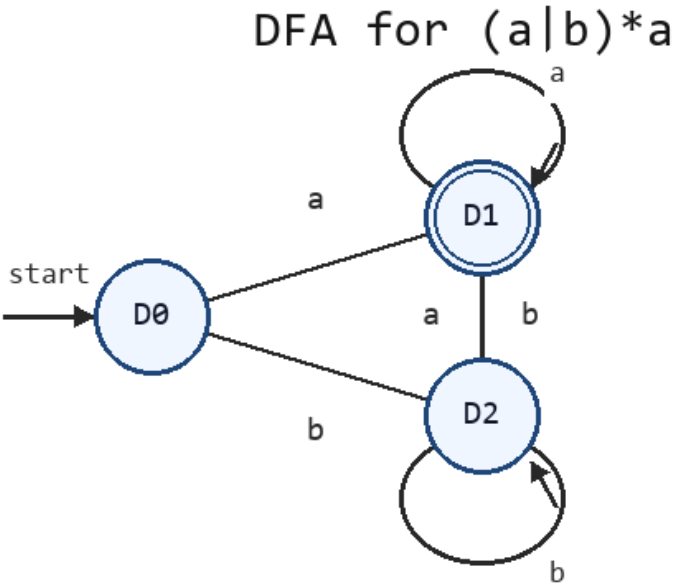
子集构造从初态闭包开始：

$D_0 = \text{epsilon-closure}(\{q_0\}) = \{q_0, q_1, q_2, q_4, q_7, q_8\}$

逐步计算：

DFA 状态	NFA 状态集合	输入 a 后	输入 b 后	是否接受
D0	{q0, q1, q2, q4, q7, q8}	D1	D2	否
D1	{q1, q2, q3, q4, q6, q7, q8, q9}	D1	D2	是
D2	{q1, q2, q4, q5, q6, q7, q8}	D1	D2	否

接受状态判断依据：NFA 状态集合中包含 q9。因此只有 D1 是 DFA 接受状态。



子集构造 DFA for $(a|b)^*a$

最小化时先按终态与非终态分组：

$P0 = \{ \{D1\}, \{D0, D2\} \}$

比较 D0 和 D2：

状态	输入 a	输入 b	结论
D0	D1	D2	a 到终态组, b 到非终态组
D2	D1	D2	a 到终态组, b 到非终态组

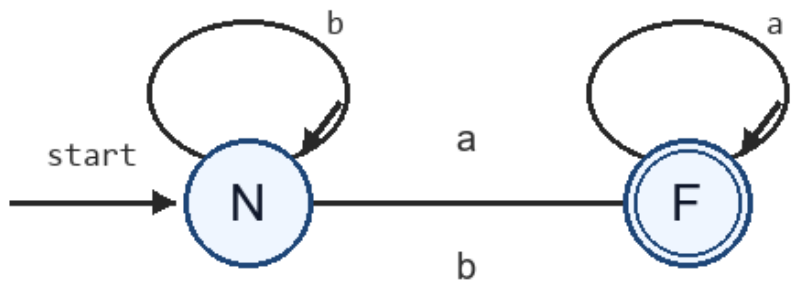
D0 和 D2 对所有输入都转到相同分组，所以等价，可以合并。令：

$N = \{D0, D2\}$
 $F = \{D1\}$

最小 DFA：

最小状态	输入 a	输入 b	是否接受
N	F	N	否
F	F	N	是

Min DFA: strings ending in a



最小 DFA for $(a|b)^*a$

(二) LL(1) 分析

原文法：

$A \rightarrow B C$
 $B \rightarrow B p \mid r$
 $C \rightarrow t \mid u$

$B \rightarrow B p \mid r$ 是直接左递归。套用：

$X \rightarrow X \alpha \mid \beta$
 $X \rightarrow \beta X'$
 $X' \rightarrow \alpha X' \mid \epsilon$

其中 $\alpha=p, \beta=r$ ，得到：

$A \rightarrow B C$
 $B \rightarrow r B'$
 $B' \rightarrow p B' \mid \epsilon$
 $C \rightarrow t \mid u$

FIRST 集：

非终结符	推导依据	FIRST	
B	$B \rightarrow r B'$	{ r }	
B'	$B' \rightarrow p B' \mid \epsilon$	ϵ	{ p, ϵ }
C	$C \rightarrow t \mid u$		{ t, u }
A	$A \rightarrow B C, B$ 不能推出 ϵ	{ r }	

FOLLOW 集：

FOLLOW 集	推导依据	结果
FOLLOW(A)	A 是开始符号	{ \$ }
FOLLOW(B)	$A \rightarrow B C, B$ 后面是 C	{ t, u }
FOLLOW(B')	$B \rightarrow r B', B'$ 在末尾	{ t, u }
FOLLOW(C)	$A \rightarrow B C, C$ 在末尾	{ \$ }

预测分析表填表依据：

产生式	依据	表项
$A \rightarrow B C$	$FIRST(BC) = \{r\}$	$M[A, r]$
$B \rightarrow r B'$	$FIRST(rB') = \{r\}$	$M[B, r]$
$B' \rightarrow p B'$	$FIRST(pB') = \{p\}$	$M[B', p]$
$B' \rightarrow \epsilon$	$FOLLOW(B') = \{t, u\}$	$M[B', t], M[B', u]$
$C \rightarrow t$	$FIRST(t) = \{t\}$	$M[C, t]$
$C \rightarrow u$	$FIRST(u) = \{u\}$	$M[C, u]$

完整预测分析表，空白处为 error：

非终结符	r	p	t	u	\$
A	$A \rightarrow B C$	error	error	error	error
B	$B \rightarrow r B'$	error	error	error	error
B'	error	$B' \rightarrow p B'$	$B' \rightarrow \epsilon$	$B' \rightarrow \epsilon$	error
C	error	error	$C \rightarrow t$	$C \rightarrow u$	error

每个表项最多一条产生式，因此改写后的文法是 LL(1)。

二、LR 分析与语义分析答案

(一) SLR 分析

增广文法：

- (0) $S' \rightarrow S$
- (1) $S \rightarrow C C$
- (2) $C \rightarrow c C$
- (3) $C \rightarrow d$

规范 LR(0) 项集族：

- I0:
 $S' \rightarrow \cdot S$
 $S \rightarrow \cdot C C$
 $C \rightarrow \cdot c C$
 $C \rightarrow \cdot d$
- I1:
 $S' \rightarrow S \cdot$
- I2:
 $S \rightarrow C \cdot C$
 $C \rightarrow \cdot c C$
 $C \rightarrow \cdot d$
- I3:
 $C \rightarrow c \cdot C$
 $C \rightarrow \cdot c C$
 $C \rightarrow \cdot d$
- I4:
 $C \rightarrow d \cdot$
- I5:
 $S \rightarrow C C \cdot$
- I6:
 $C \rightarrow c C \cdot$

GOTO 转移表：

来源	符号	目标
I0	S	I1
I0	C	I2
I0	c	I3
I0	d	I4
I2	C	I5
I2	c	I3
I2	d	I4
I3	C	I6
I3	c	I3
I3	d	I4

FOLLOW:

非终结符	推导依据	FOLLOW
S	S 是开始符号	{ \$ }
C	S $\rightarrow$ C C: 第一个 C 后接 C, 加入 FIRST(C)={c,d}; 第二个 C 在末尾, 加入 FOLLOW(S)	{ c, d, \$ }

SLR 填表规则:

点前进到终结符时填 shift。
完成项 A $\rightarrow$ alpha . 只在 FOLLOW(A) 上填 reduce。
S' $\rightarrow$ S . 在 \$ 上填 acc。

完整 SLR ACTION/GOTO 表:

状态	ACTION(c)	ACTION(d)	ACTION(\$)	GOTO(S)	GOTO(C)
0	s3	s4	error	1	2
1	error	error	acc	error	error
2	s3	s4	error	error	5
3	s3	s4	error	error	6
4	r3	r3	r3	error	error
5	error	error	r1	error	error
6	r2	r2	r2	error	error

冲突判断: 表中没有任何单元格同时出现 shift 和 reduce, 也没有任何单元格同时出现两个 reduce, 所以不存在移进-规约冲突或规约-规约冲突。

(二) L 属性 SDD

设计思路: T.type 是类型来源, 向右传给 L.in 和 L'.in, 每遇到一个 id 就把该名字和传入类型加入符号表。

产生式	语义规则或动作
D $\rightarrow$ T L	L.in = T.type
T $\rightarrow$ int	T.type = int
T $\rightarrow$ float	T.type = float
L $\rightarrow$ id L'	addType(id.name, L.in); L'.in = L.in
L' $\rightarrow$, id L'1	addType(id.name, L'.in); L'1.in = L'.in
L' $\rightarrow$ epsilon	不执行插入动作

输入 float x, y 的属性传递:

步骤	动作	结果
1	T $\rightarrow$ float	T.type=float
2	D $\rightarrow$ T L	L.in=float

步骤	动作	结果
3	识别 x	addType(x, float)
4	进入 L'	L'.in=float
5	识别 y	addType(y, float)

最终符号表：

名字	类型
x	float
y	float

继承属性只依赖父结点或左侧兄弟结点，因此满足 L 属性限制。

三、中间代码、控制流与 SSA 答案

1. 三地址码

数组元素宽度为 4，a[i] 的地址偏移为 i * 4。

```

L1:
  if i < n goto L2
  goto L6

L2:
  t1 = i * 4
  t2 = a[t1]
  if t2 > 0 goto L3
  goto L4

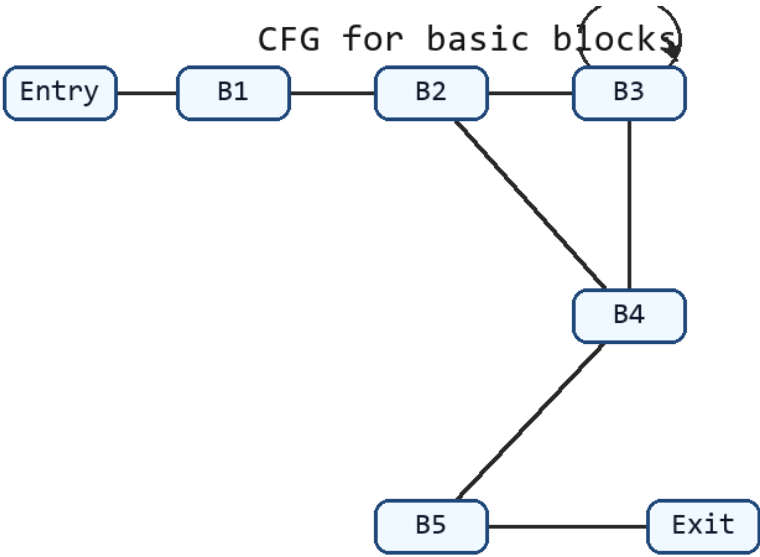
L3:
  sum = sum + t2
  goto L4

L4:
  i = i + 1
  goto L1

L6:

```

2. 基本块与 CFG



基本块	语句
B1	L1 条件判断，真到 B2，假到 B5
B2	数组地址计算、读取和 $a[i] > 0$ 判断
B3	$sum = sum + t2$
B4	$i = i + 1$ ，回到 B1
B5	L6 循环出口

CFG 边：

```

B1 -> B2
B1 -> B5
B2 -> B3
B2 -> B4
B3 -> B4
B4 -> B1

```

3. SSA

循环头 B1 有来自循环外和回边 B4 的两条路径，因此 i 和 sum 都需要 phi。B4 还有来自 B2 假分支和 B3 真分支的汇合，因此 sum 在 B4 也需要 phi。

```

L1:
i1 = phi(i0, i2)
sum1 = phi(sum0, sum3)
if i1 < n goto L2
goto L6

```

```

L2:
t1 = i1 * 4
t2 = a[t1]
if t2 > 0 goto L3
goto L4

```

```

L3:
sum2 = sum1 + t2
goto L4

```

```

L4:
sum3 = phi(sum1, sum2)
i2 = i1 + 1
goto L1

```

```

L6:

```

4. 退出 SSA 的复制边

phi 展开原则：在每条前驱边上插入对应复制。

phi	前驱边	插入复制
$i1 = \text{phi}(i0, i2)$	循环外 -> L1	$i1 = i0$
$i1 = \text{phi}(i0, i2)$	L4 -> L1	$i1 = i2$
$sum1 = \text{phi}(sum0, sum3)$	循环外 -> L1	$sum1 = sum0$
$sum1 = \text{phi}(sum0, sum3)$	L4 -> L1	$sum1 = sum3$
$sum3 = \text{phi}(sum1, sum2)$	L2 -> L4	$sum3 = sum1$
$sum3 = \text{phi}(sum1, sum2)$	L3 -> L4	$sum3 = sum2$

这些复制在语义上是并行复制；若控制流边不能直接插入指令，需要拆分边。

四、运行时环境与目标代码生成答案

（一）运行时环境

add 的活动记录至少包含：

内容	作用
参数 a、b	被调用函数读取实参
局部变量 t	保存 a+b 的局部结果
返回地址	指出 add 返回后继续执行的位置
旧 BP / 控制链	返回时恢复调用者活动记录
保存的寄存器	保护调用前的机器状态
返回值位置	按调用约定传回 t

参数从右到左压栈，调用 add(x, y) 的顺序是：

顺序	动作
1	压入 y 的值，作为形参 b
2	压入 x 的值，作为形参 a
3	执行 call add，保存返回地址
4	被调函数建立新活动记录

BP、SP、返回地址、返回值作用：

项	作用
SP	指向当前栈顶，压栈和退栈会改变 SP
BP	当前活动记录的稳定基址，用固定偏移访问参数和局部变量
返回地址	被调函数结束后跳回调用点
返回值	放入约定寄存器或栈中约定位置，供调用者读取

垃圾回收比较：

方法	优点	缺点
引用计数	对象计数变为 0 时可立即回收	无法回收不可达环
标记-清扫	从根集标记可达对象，能回收不可达环	需要遍历堆，可能产生暂停

引用计数无法处理的结构是不可达循环引用，例如两个对象互相引用但已经没有根指向它们。

（二）目标代码生成

三地址码：

```
t1 = a - b
t2 = t1 + c
x = t2
```

目标代码：

```
LD R0, a
LD R1, b
SUB R0, R1
LD R1, c
ADD R0, R1
ST x, R0
```

寄存器描述符变化：

步骤	指令	R0	R1
初始	-	{}	{}
1	LD R0, a	{a}	{}

步骤	指令	R0	R1
2	LD R1, b	{a}	{b}
3	SUB R0, R1	{t1}	{b}
4	LD R1, c	{t1}	{c}
5	ADD R0, R1	{t2}	{c}
6	ST x, R0	{t2, x}	{c}

最终地址描述符中：

$Addr(x) = \{\text{内存位置 } x, R0\}$

五、数据流分析、支配与自然循环答案

1. 到达定值分析框架

到达定值是正向 may 分析：

$IN[B] = \text{union } OUT[P], P \text{ 是 } B \text{ 的所有前驱}$
 $OUT[B] = \text{gen}[B] \text{ union } (IN[B] - \text{kill}[B])$

初始化按题目要求从空集开始。

2. gen/kill

所有定义：

$d1: x = 1$
 $d2: y = 2$
 $d3: z = x + y$
 $d4: x = z$

d1 和 d4 都定义 x，互相 kill。

基本块	gen	kill
B1	{d1, d2}	{d4}
B2	{d3}	{}
B3	{d4}	{d1}
B4	{}	{}

3. IN/OUT 不动点结果

基本块	IN	OUT
B1	{}	{d1, d2}
B2	{d1, d2, d3, d4}	{d1, d2, d3, d4}
B3	{d1, d2, d3, d4}	{d2, d3, d4}
B4	{d1, d2, d3, d4}	{d1, d2, d3, d4}

关键推导：

$IN[B2] = OUT[B1] \text{ union } OUT[B3]$
 $= \{d1, d2\} \text{ union } \{d2, d3, d4\}$
 $= \{d1, d2, d3, d4\}$

所以循环回边使 d3、d4 也能到达 B2。

4. 支配集合和直接支配者

支配定义：从 Entry 到某节点的每条路径都经过的节点，才支配该节点。

节点	Dom
Entry	{Entry}
B1	{Entry, B1}
B2	{Entry, B1, B2}
B3	{Entry, B1, B2, B3}
B4	{Entry, B1, B2, B4}
Exit	{Entry, B1, B2, B4, Exit}

直接支配者：

节点	idom
B1	Entry
B2	B1
B3	B2
B4	B2
Exit	B4

5. 回边与自然循环

回边判断：若边 $n \rightarrow h$ 中 h 支配 n ，则该边是回边。

CFG 中：

$B3 \rightarrow B2$

因为 $B2$ 支配 $B3$ ，所以 $B3 \rightarrow B2$ 是回边。

自然循环构造：从回边尾 $B3$ 反向追溯前驱，直到回边头 $B2$ ，并包含头和尾：

自然循环 = { $B2, B3$ }